UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Calidad de Software

Mag. Pablo Virgolini

10 de agosto de 2026

¿Recordamos?



¿Qué es la Calidad de Software?

Calidad de software

Capacidad del software para cumplir con los requisitos especificados, satisfacer las expectativas del usuario, ofrecer un rendimiento eficiente y ser mantenible en el tiempo.

Conformidad

Cumplimiento de los requisitos técnicos y funcionales establecidos.

Satisfacción

Capacidad de satisfacer las expectativas y necesidades del usuario.

Eficiencia

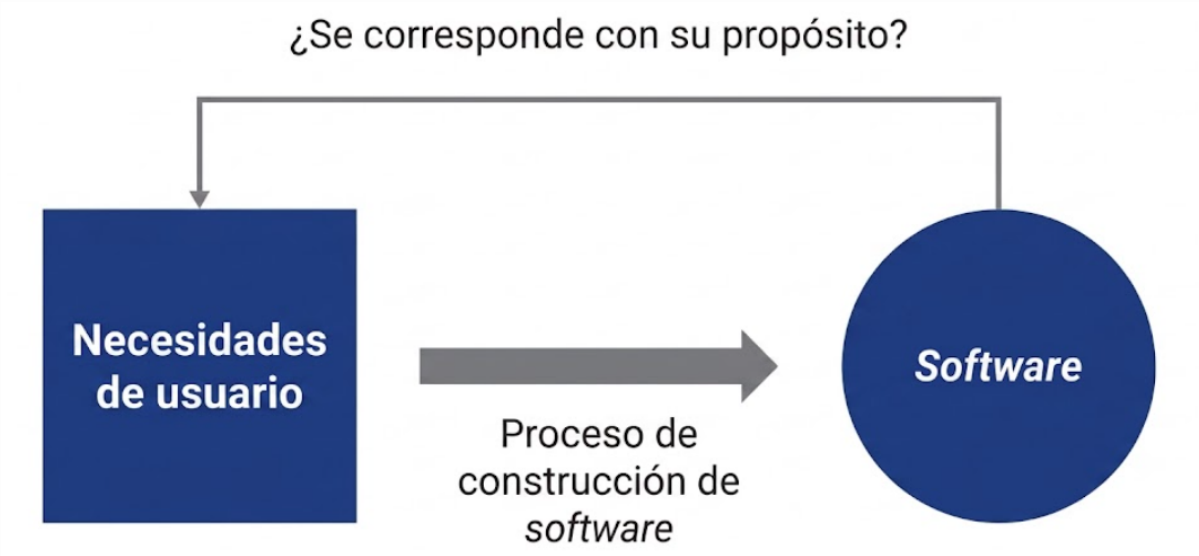
Uso óptimo de los recursos disponibles con buen rendimiento.

Mantenibilidad

Facilidad para corregir, actualizar y mejorar el software.

Software de calidad

La **corrección** de un programa se define en relación a las necesidades del usuario, por lo que fundamental que se cumpla para que sea útil.



Software de calidad

El siguiente programa está escrito en Python y su sintaxis es válida.

```
def procesar_array(tabla):
    i = 0
    while i < len(tabla) and tabla[i] != 0:
        i = i + 1
    if i < len(tabla):
        return i
    else:
        return -1


if __name__ == "__main__":
    # Ejemplos de prueba
    ejemplo1 = [5, 3, 8, 0, 2]
    ejemplo2 = [5, 3, 8, 2]


    print(f"procesar_array({ejemplo1}) -> {procesar_array(ejemplo1)}")
    print(f"procesar_array({ejemplo2}) -> {procesar_array(ejemplo2)}")
```

¿Es correcto?

Software de calidad



El programa es parte de la implementación de un juego de ingenio que la empresa está por lanzar. **No es suficiente**



El programa procesarArray encuentra la primera posición libre en un arreglo.

¿Qué significa que una posición del arreglo esté libre?



El programa procesarArray encuentra la primera posición libre de un arreglo. El valor cero en la posición indica que la misma está libre, mientras que un valor distinto de cero indica que la posición está ocupada. Si todas las posiciones del arreglo están ocupadas, el programa debe retornar -1.

Especificación

La **especificación** es una descripción abstracta del diseño del software que es una base para un diseño e implementación detallados.



Verificación y validación

Verificación

¿Construimos bien el producto?

Controla que el software cumple con lo establecido en la especificación del usuario (funcional y no funcional).

Enfoque: Comparar el producto contra la especificación.

Validación

¿Construimos el producto correcto?

Lo especificado puede no representar las necesidades reales; la validación asegura que el producto cumple con las necesidades del cliente.

Enfoque: Comparar el producto contra las necesidades del cliente.



Verificación

Cumplimiento de Especificación

Evalúa código, arquitectura y diseños sin necesidad de ejecutar todo el sistema.

Actividades Estáticas

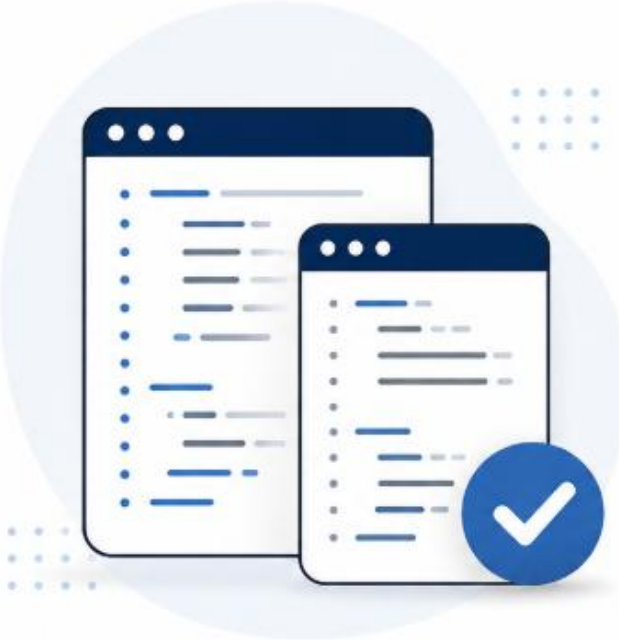
Incluye revisiones de código, inspecciones de diseño y análisis estático de requerimientos.

Pruebas Unitarias e Integración

Comprueba que los bloques individuales funcionan según su diseño técnico.

Objetivo Principal

Encontrar fallas internas de desarrollo de forma temprana.



Validación

Satisfacción del Cliente

Confirma que el producto resuelve el problema real del usuario final.

Actividades Dinámicas

Requiere ejecutar el software en entornos reales o simulados.

Pruebas de Aceptación (UAT)

Pruebas de usabilidad y validación directa por parte del cliente o stakeholders.

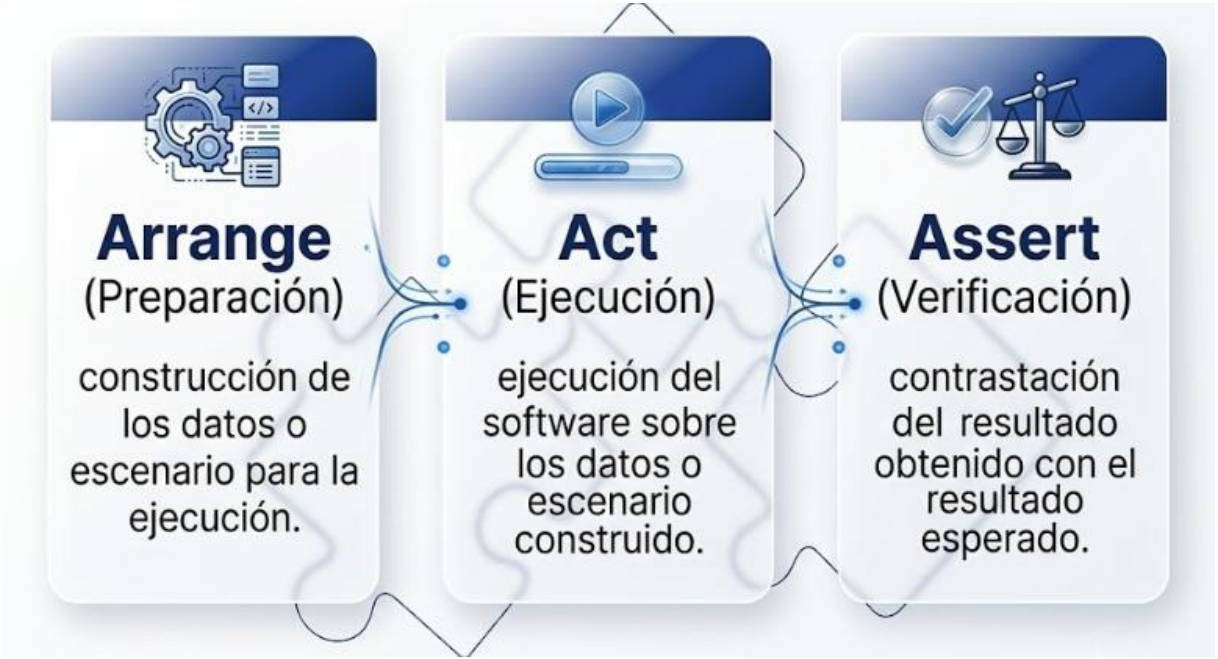
Objetivo Principal

Garantizar el valor de negocio y la utilidad del software.



Componentes de un test de software

El testing de verificación comprueba que el software cumple con la función descripta en su especificación, para un conjunto de escenarios de ejecución. Esto da lugar a tres componentes fundamentales.



Assert

Es una instrucción que verifica una condición y corta el programa si no se cumple lo esperado.

```
assert condicion, "mensaje opcional"

assert procesar_array([]) == -1, "Falló: arreglo vacío"
```



condición

La expresión que tiene que dar True. Si da False, el assert falla.

mensaje (opcional)

Texto que se muestra junto al error, para saber qué caso rompió sin adivinar.

AssertionError

La excepción que Python lanza automáticamente cuando la condición es False.

Ejemplo aplicado a *procesar_array*

Aplicamos Arrange – Act – Assert sobre un caso concreto

1

Arrange

Preparamos el escenario: un arreglo con un 0 en la posición 3.

```
tabla = [5, 3, 8, 0, 2]
```

2

Act

Ejecutamos la función sobre ese escenario.

```
resultado = procesar_array(tabla)
```

3

Assert

Contrastamos el resultado contra lo esperado.

```
assert resultado == 3, "Falló: 0 en el medio"
```

Si procesar_array cambia y deja de cumplir esto, el assert lo va a detectar.

Caso: Incidente VSE

▶ Intro
Te invito a leer el caso de la Vancouver Stock Exchange (VSE) .

▶ Mecanismo del Bug
El sistema truncaba el índice a 3 decimales después de cada recálculo, en vez de conservar la precisión completa y redondear recién al mostrar el valor final. Como cada recálculo partía del valor ya truncado (y no del real), el error no se descartaba: se arrastraba al siguiente cálculo, siempre en la misma dirección.

▶ Pregunta
¿Por qué un error tan "chico" como un truncamiento en cada paso puede hacer que un índice pierda el 70% de su valor real?

El incidente real VSE

La bolsa de Vancouver truncaba el índice a **3 decimales** después de cada recálculo, en vez de conservar la precisión completa y redondear recién al mostrar el valor final.



El mecanismo

Cada recalculo partía del valor ya truncado, no del real: el error de cada paso se arrastraba al siguiente, siempre hacia abajo.



La escala

Miles de operaciones por día, durante casi 2 años. Un error mínimo por operación, repetido sin descanso.



El resultado

El índice llegó a mostrar +524 puntos, cuando el valor correcto era +1098: más del doble. Una pérdida sistemática, no ruido de mercado.



Para que el error se acumule siempre en la misma dirección, tiene que ser un **truncamiento sistemático** (redondeo real – el más cercano – es estadísticamente imparcial y no lo reproduce).